

RAG系统： 知识检索增强

生成之前，先检索 —— 检索、增强、生成。

主讲人：柠檬酸

什么是 RAG

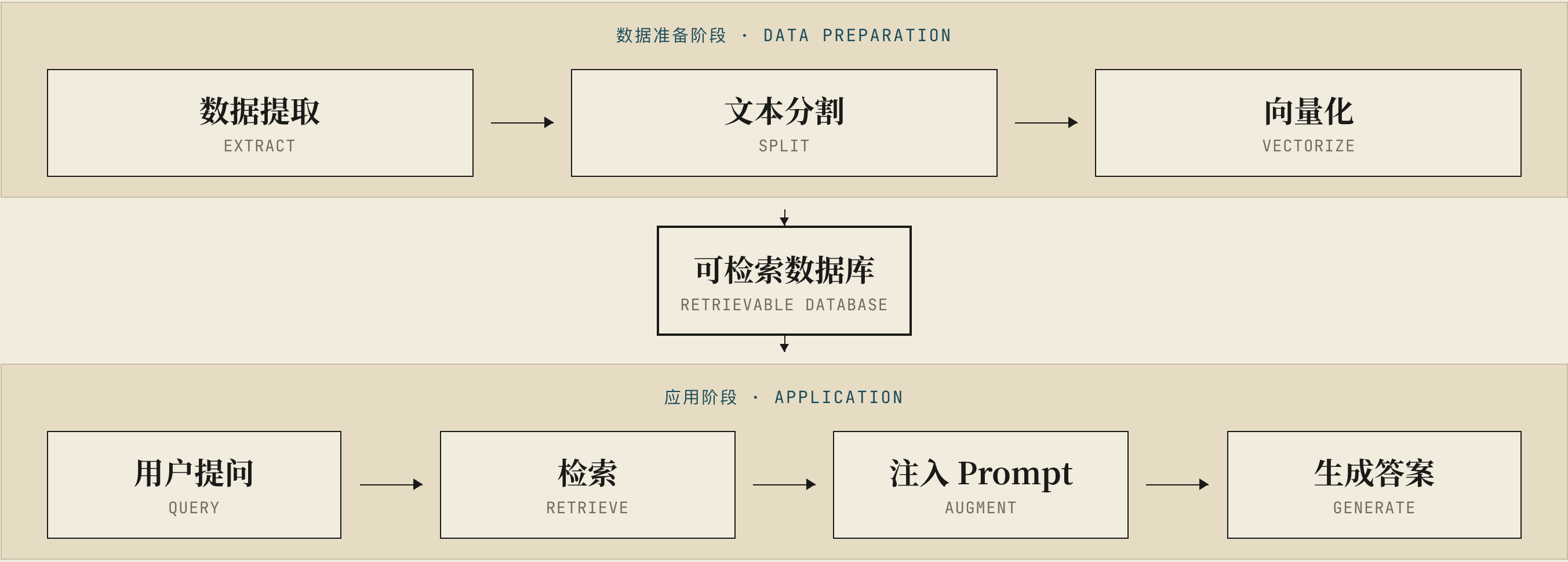
检索增强生成（Retrieval-Augmented Generation）—— 一种结合了信息检索和文本生成的技术

RETRIEVAL · 检索	AUGMENTED · 增强	GENERATION · 生成
从知识库中查询相关内容——生成回答之前，先从外部知识库检索相关信息	将检索到的信息作为上下文融入提示词，提供给大语言模型，辅助模型生成	基于增强后的上下文作答，输出兼具准确性与透明度的答案
外部知识库 · 为模型提供事实依据	检索结果 · 注入 Prompt	更准确 · 更可靠的回答

RAG = 检索 Retrieval + 增强 Augmented + 生成 Generation

基本工作流程：两大核心环节

数据准备阶段构建可检索的数据库，应用阶段响应提问、生成答案



数据准备：提取 → 分割 → 向量化 | 应用：提问 → 检索 → 注入 Prompt → 生成答案

发展历程：三个阶段

从「检索-读取」模式，到模块化、自动化和智能化

2020 - 2021 · NAIVE RAG

朴素 RAG

技术的萌芽阶段，流程直接而简单，被称为「检索-读取」（Retrieve-Read）模式：计算词频与文档频率评估相关性，字面匹配效果好，但难以理解语义上的相似性

检索 TF-IDF / BM25 关键词匹配
生成 文档直接拼接进提示词上下文

2022 - 2023 · ADVANCED RAG

高级 RAG

向量数据库和文本嵌入技术成熟，在「检索」和「生成」的各个环节引入大量优化技术：文本转为高维向量后，模型能够理解和匹配语义上的相似性

检索 稠密嵌入 Dense Embedding 语义检索
生成 查询重写 · 文档分块 · 重排序

2023 - 至今 · MODULAR RAG

模块化 RAG

系统的各个部分被设计成可插拔、可组合的独立模块，向着模块化、自动化和智能化的方向发展，适应更多样化和复杂的应用场景

检索 混合检索 · 多查询扩展 · 假设性文档嵌入
生成 思维链推理 · 自我反思与修正

RAG 系统的两个主要工作模式



设计思路：将传入的一切外部知识源，统一转化为 Markdown 格式进行处理

快速体验 RAG 功能

先跑起来，再深入原理

```
# 创建 LLM 与 RAG 工具（知识库路径 / 集合名 / 命名空间）
llm = HelloAgentsLLM()
rag_tool = RAGTool(knowledge_base_path="./knowledge_base",
                   collection_name="test_collection", rag_namespace="test")

# 注册工具，创建具有 RAG 能力的 Agent
tool_registry = ToolRegistry()
tool_registry.register_tool(rag_tool)
agent = SimpleAgent(name="知识助手", llm=llm, tool_registry=tool_registry)

# 添加知识：文本内容 + 文档 ID
rag_tool.execute("add_text", text="Python是一种高级编程语言.....", document_id="python_intro")

# 检索知识 / 查看知识库统计
rag_tool.execute("search", query="Python编程语言的历史", limit=3, min_score=0.1)
rag_tool.execute("stats")
```

add_text

添加知识：传入文本内容，document_id 标记文档身份

search

检索知识：limit 控制返回条数，min_score 过滤低分结果

stats

知识库统计：查看当前知识库的规模与状态

「五层七步」的分层设计

每一层都可以独立优化和替换，同时保持整体系统的稳定性



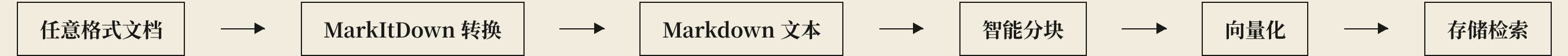
可插拔：嵌入模型从 sentence-transformers 切换到百炼 API，上层业务逻辑零改动

RAGTool: RAG 系统的统一入口

完整的 RAG 能力：多格式文档添加 · 智能检索召回 · LLM 增强问答 · 知识库管理

```
class RAGTool(Tool):
    """RAG工具：多格式文档添加 / 智能检索召回 / LLM增强问答 / 知识库管理"""
    def __init__(self,
        knowledge_base_path: str = "./knowledge_base", # 知识存储路径
        qdrant_url: str = None, # 向量数据库连接
        collection_name: str = "rag_knowledge_base",
        rag_namespace: str = "default"): # 命名空间：多知识库隔离
        self._pipelines = {} # 命名空间 → RAG 管道
        self.llm = HelloAgentsLLM() # 增强问答所需的 LLM
        # 创建默认管道：每个命名空间对应一条独立的 RAG pipeline
        default_pipeline = create_rag_pipeline(
            qdrant_url=self.qdrant_url, collection_name=self.collection_name,
            rag_namespace=self.rag_namespace)
        self._pipelines[self.rag_namespace] = default_pipeline
```

完整实现：RAGTool 类定义与默认管道初始化



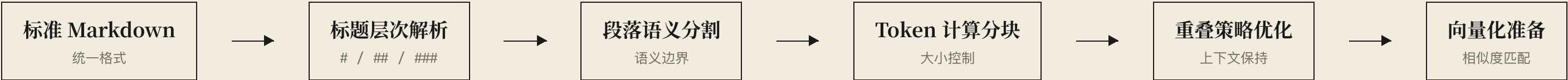
MarkItDown：统一文档转换引擎

微软开源的通用文档转换工具——无论输入是什么格式，最终都转换为标准的 Markdown 文本

文档	图像	音频	文本	代码
PDF · Word · Excel · PPT	JPG · PNG · GIF (OCR)	MP3 · WAV · M4A (转录)	TXT · CSV · JSON · XML · HTML	Python · JavaScript · Java 等

```
def _convert_to_markdown(path: str) -> str:
    """核心功能：将任意格式文档转换为 Markdown 文本"""
    if not os.path.exists(path):
        return ""
    ext = (os.path.splitext(path)[1] or '').lower()
    if ext == '.pdf':
        return _enhanced_pdf_processing(path)    # PDF 走增强处理
    md_instance = _get_markitdown_instance()      # 其他格式：MarkItDown 统一转换
    if md_instance is None:
        return _fallback_text_reader(path)        # 兜底：纯文本读取器
    result = md_instance.convert(path)
    markdown_text = getattr(result, "text_content", None)
    if isinstance(markdown_text, str) and markdown_text.strip():
        return markdown_text                      # 转换成功，返回 Markdown
    return ""
```

结构感知：按标题层次分割段落



```
def _split_paragraphs_with_headings(text: str) -> List[Dict]:
    """根据标题层次分割段落，保持语义完整性"""
    heading_stack, paragraphs, buf = [], [], []
    def flush_buf(end_pos):
        # 将缓冲区落为一个段落
        content = "\n".join(buf).strip()
        if content: paragraphs.append({ "content": content,
            "heading_path": " > ".join(heading_stack) or None, # 记录标题路径
            "start": ..., "end": end_pos })
    for raw in text.splitlines():
        if raw.strip().startswith("#"):
            # 标题行：落盘并维护标题栈
            flush_buf(...)
            level = len(raw) - len(raw.lstrip('#'))
            heading_stack = heading_stack[:level-1] + [raw.lstrip('#').strip()]
        elif raw.strip() == "":
            # 空行 = 段落边界
            flush_buf(...); buf = []
        else: buf.append(raw)
            # 普通行：继续累积
    return paragraphs
```

完整实现：_split_paragraphs_with_headings（标题栈维护 / 段落落盘 / 字符位置记录）

Token 计算分块与中英文估算

① `_chunk_paragraphs` · 基于 Token 数量的智能分块

```
def _chunk_paragraphs(paragraphs, chunk_tokens, overlap_tokens):
    """基于 Token 数量的智能分块"""
    chunks, cur, cur_tokens, i = [], [], 0, 0
    while i < len(paragraphs):
        p_tokens = _approx_token_len(paragraphs[i]["content"]) or 1
        if cur_tokens + p_tokens ≤ chunk_tokens or not cur:
            cur.append(paragraphs[i]); cur_tokens += p_tokens; i += 1 # 装得下: 累积
            continue
        # 超预算: 产出当前分块 (取最近段落的标题路径)
        chunks.append({ "content": "\n\n".join(x["content"] for x in cur),
            "start": cur[0]["start"], "end": cur[-1]["end"],
            "heading_path": next((x["heading_path"] for x in reversed(cur)
                if x.get("heading_path")), None) })
        kept, kept_tokens = [], 0 # 重叠: 从尾部回留若干段落
        for x in reversed(cur):
            t = _approx_token_len(x["content"]) or 1
            if kept_tokens + t > overlap_tokens: break
            kept.append(x); kept_tokens += t
        cur, cur_tokens = list(reversed(kept)), kept_tokens
    return chunks # 循环后补收最后一个分块
```

② `_approx_token_len` · 中英文混合 Token 估算

```
def _approx_token_len(text: str) → int:
    """近似估计 Token 长度, 支持中英文混合"""
    cjk = sum(1 for ch in text if _is_cjk(ch)) # CJK 按 1 token 计
    non_cjk = len([t for t in text.split() if t]) # 其余按空白分词
    return cjk + non_cjk

def _is_cjk(ch: str) → bool:
    """判断是否为 CJK 字符 (查 Unicode 码位区间) """
    code = ord(ch)
    return (0x4E00 ≤ code ≤ 0x9FFF or # CJK 统一汉字
        0x3400 ≤ code ≤ 0x4DBF or # CJK 扩展 A
        0x20000 ≤ code ≤ 0x2A6DF or # CJK 扩展 B
        0xF900 ≤ code ≤ 0xFAFF) # CJK 兼容汉字
```

CJK = Chinese · Japanese · Korean

中日韩统一表意文字, 核心为汉字, 在 Unicode 中占专门码位区间。汉字无空格, 逐字按 1 token 计; 英文按空白分词计数——两段相加即为混合文本的近似 token 长度, 分块大小才控得准。

完整实现: `_chunk_paragraphs` 分块主循环 (含末块补收) + 七段 CJK 码位表

统一嵌入接口与向量存储



```
def index_chunks(store=None, chunks=None, batch_size=64, rag_namespace="default"):
    """统一嵌入 + Qdrant 存储: 百炼 API, 回退 sentence-transformers"""
    if not chunks: return
    embedder = get_text_embedder()          # 统一嵌入器 (内部处理缓存)
    dimension = get_dimension(384)
    if store is None:                        # 默认创建 Qdrant 向量库
        store = _create_default_vector_store(dimension)
    # 预处理 Markdown 文本, 获得更好的嵌入质量
    processed = [_preprocess_markdown_for_embedding(c["content"]) for c in chunks]
    vecs = []
    for i in range(0, len(processed), batch_size): # 批量编码
        part_vecs = embedder.encode(processed[i:i+batch_size])
        for v in part_vecs:
            v_norm = [float(x) for x in v]
            if len(v_norm) != dimension:          # 维度对齐: 补零 / 截断
                v_norm = (v_norm + [0.0] * dimension)[:dimension]
            vecs.append(v_norm)
```

三种互补的高级检索策略

用户的查询表述与文档中的实际内容可能存在用词差异，导致相关文档无法被检索到

STRATEGY 1 · MQE

多查询扩展

用 LLM 生成语义等价的多样化查询，并行执行这些扩展查询并合并结果，覆盖更广泛的相关文档

解决：用词多样性
对模糊查询 / 专业术语查询效果显著

STRATEGY 2 · HYDE

假设文档嵌入

让 LLM 先生成一个假设性的答案段落，再用这个答案段落去检索真实文档——「用答案找答案」

解决：语义鸿沟
假设答案与真实答案语义更接近

STRATEGY 3 · FRAMEWORK

统一扩展检索框架

将 MQE 和 HyDE 整合到统一框架中，通过 `enable_mqe / enable_hyde` 参数按场景选择启用

扩展 → 检索 → 合并 三步流程
`candidate_pool_multiplier = 4` 扩大候选池

MQE：一个问题的多种表述

同一个问题可以有多种不同的表述方式，而不同的表述可能匹配到不同的相关文档

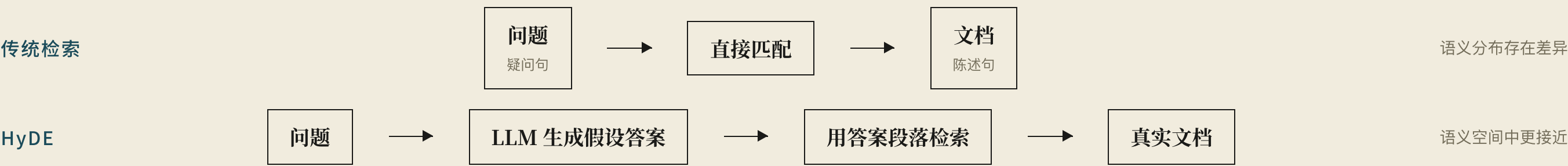
「如何学习Python」 ⇒ 「Python入门教程」·「Python学习方法」·「Python编程指南」

```
def _prompt_mqe(query: str, n: int) → List[str]:
    """使用 LLM 生成多样化的查询扩展"""
    try:
        from ...core.llm import HelloAgentsLLM
        llm = HelloAgentsLLM()
        prompt = [
            {"role": "system", "content": "你是检索查询扩展助手。生成语义等价或互补的多样化查询。使用中文，简短，避免标点。"},
            {"role": "user", "content": f"原始查询: {query}\n请给出{n}个不同表述的查询，每行一个。"}
        ]
        text = llm.invoke(prompt)          # 调用 LLM 生成扩展
        lines = [ln.strip("- \t") for ln in (text or "").splitlines()]
        outs = [ln for ln in lines if ln]
        return outs[:n] or [query]        # 取前 n 个；解析为空时退回原查询
    except Exception:
        return [query]                    # 异常兜底：至少保留原查询
```

完整实现：_prompt_mqe (LLM 生成扩展查询 + 逐行解析 + 异常兜底)

HyDE：用答案找答案

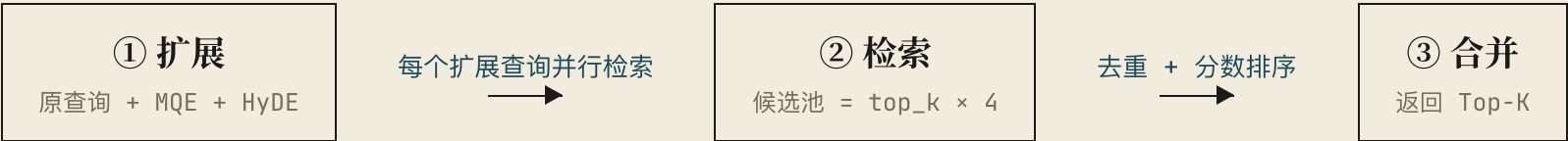
问题通常是疑问句，而文档内容是陈述句——问题和答案在语义空间中的分布往往存在差异



```
def _prompt_hyde(query: str) → Optional[str]:
    """生成假设性文档用于改善检索"""
    try:
        from ...core.llm import HelloAgentsLLM
        llm = HelloAgentsLLM()
        prompt = [
            {"role": "system", "content": "根据用户问题，先写一段可能的答案性段落，用于向量检索的查询文档（不要分析过程）。"},
            {"role": "user", "content": f"问题: {query}\n请直接写一段中等长度、客观、包含关键术语的段落。"}
        ]
        return llm.invoke(prompt) # 假设答案即使不完全正确，关键术语也能引导检索
    except Exception:
        return None
```

完整实现：_prompt_hyde（假设性文档生成 · 专业领域查询效果显著）

统一框架：扩展 - 检索 - 合并



```
def search_vectors_expanded(store=None, query="", top_k=8, enable_mqe=False,
                             mqe_expansions=2, enable_hyde=False, candidate_pool_multiplier=4, ...):
    """扩展-检索-合并：统一查询扩展检索"""
    if not query: return []
    expansions = [query] # ① 扩展：原查询 + MQE + HyDE
    if enable_mqe and mqe_expansions > 0: expansions.extend(_prompt_mqe(query, mqe_expansions))
    if enable_hyde and (t := _prompt_hyde(query)): expansions.append(t)
    expansions = list(dict.fromkeys(expansions)) # 去重保序
    pool = max(top_k * candidate_pool_multiplier, 20) # ② 候选池 ×4 扩大
    per = max(1, pool // max(1, len(expansions))) # 每个扩展查询的检索额度
    agg = {} # ③ 合并：按文档 ID 去重留高分
    for q in expansions:
        hits = store.search_similar(query_vector=embed_query(q), limit=per, ...)
        for h in hits:
            mid = h.get("metadata", {}).get("memory_id", h.get("id"))
            if mid not in agg or h["score"] > agg[mid]["score"]: agg[mid] = h
    return sorted(agg.values(), key=lambda x: x["score"], reverse=True)[:top_k]
```

完整实现：search_vectors_expanded（命名空间过滤 / 候选池分配 / 聚合排序）

如何选择检索策略

还有很多有趣的方法——在实际的使用场景里，去尝试寻找适合问题的解决方案

一般查询

启用 MQE

多查询扩展覆盖用词多样性，避免因用词差异遗漏重要信息

推荐 `enable_mqe=True`

专业领域查询

MQE + HyDE

HyDE 生成包含领域术语的假设文档，显著提升检索精度

推荐 同时启用 `enable_mqe + enable_hyde`

性能敏感场景

基础检索

跳过 LLM 扩展调用，延迟最低；也可以只使用基础检索或仅启用 MQE

推荐 基础检索 / 仅 MQE

本章回顾 · MarkItDown 统一载入 → 结构感知分块 → 统一嵌入存储 → MQE / HyDE → 扩展-检索-合并